

Class Extension Report

In this task, the data model from Assessment 1 and 2 was changed more so the banking app can do the new work properly. The main need was to support a one-to-many relationship. This means one customer can now have many accounts, not just one. Another important change was to separate a normal customer and a bank staff customer, because staff customers get a 50% discount on failed transaction fees. So the classes had to be extended for this new logic.

The main classes changed in this task were the Customer class, the Account class, and the BankController class. These classes now work together to manage customers, accounts, transfers, balances, and other operations in the prototype.

Customer Class Updates

The Customer class was extended so it can hold more than one account. In Assessment 1, the customer details were basic. In this enhanced version, the customer still stores the main identity details, but now it also stores a collection of accounts. This is needed because the app allows one customer to have an Everyday account, an Investment account, and an Omni account at the same time.

The attributes in the Customer class are:

- customerNumber
- name
- contact
- isStaff
- accounts

The customerNumber is used as the unique id for the customer. The name stores the customer name. The contact stores contact details like email or phone. The isStaff value is very important because it tells the system if the customer is a bank staff member or not. The accounts list stores all account objects linked to that customer.

The methods in the Customer class are:

- GetCustomerNumber()
- GetName()
- GetContact()
- IsStaff()
- SetName()

- SetContact()
- SetStaff()
- AddAccount()
- GetAccounts()
- FindAccount()

These methods allow the program to read and update customer details. They also allow accounts to be added and searched. The AddAccount() method is very important because it supports the one-to-many structure. The FindAccount() method helps the controller locate the selected account for transfer, deposit, withdraw, or interest actions.

Account Class Updates

The Account class is the main parent class for all account types. It stays as an abstract class because the program should not create a general account directly. Instead, the child classes inherit from it. These child classes are EverydayAccount, InvestmentAccount, and OmniAccount.

The attributes in the Account class are:

- accountID
- balance
- interestRate
- overdraftLimit
- failedFee
- last

The accountID stores the unique account number. The balance stores the money in the account. The interestRate stores the account interest rule. The overdraftLimit is mainly for the Omni account. The failedFee stores the fee used when a transaction fails. The last variable stores the last message shown to the user, so the program can display the result of the latest action.

The methods in the Account class are:

- GetAccountID()
- GetBalance()
- GetInterestRate()
- GetOverdraftLimit()
- GetFailedFee()
- Last()
- GetAccountType()

- Deposit()
- Withdraw()
- CalculateInterest()
- AddInterest()
- ToString()

The Deposit() and Withdraw() methods are central to the banking app. The Withdraw() method also supports failed fee logic, and it checks if the customer is staff so the staff discount can apply. The CalculateInterest() method is abstract, because every account type uses a different rule. The AddInterest() method is a helper method which adds the interest value into the balance.

This class was not only storing shared data, but also reducing repetition. That is why inheritance was useful here.

Child Account Classes

The child account classes extend the Account class and use different rules.

The EverydayAccount is the simplest one. It has no interest and no overdraft. It only uses the base features from Account. Its CalculateInterest() method adds zero interest.

The InvestmentAccount has interest, but no overdraft. In this project, interest is applied when the balance is above the required amount. It also has failed fee logic when a withdrawal or transfer cannot be completed. If the customer is staff, the failed fee is reduced by 50%.

The OmniAccount has both interest and overdraft support. It can allow transfers or withdrawals beyond the balance up to the overdraft limit. But if the amount goes beyond the allowed limit, the failed fee is charged. It also uses the staff discount rule. The Omni account also has its own interest calculation rule.

These child classes make the system more realistic because different account types do not behave the same way.

Controller Class Updates

The controller was very important in this task. In this project the controller class is called BankController. It acts like the middle part between the forms and the model classes. So the Windows Forms do not directly handle all banking logic by themselves. This made the design cleaner.

The attributes in the BankController class are:

- customers
- nextAccountNumber

The customers list stores all customer objects in the system. The nextAccountNumber is used to generate new account ids when a new account is created.

The methods in the BankController class are:

- GetCustomers()
- FindCustomer()
- AddCustomer()
- UpdateCustomer()
- DeleteCustomer()
- AddAccountToCustomer()
- GetCustomerAccounts()
- Deposit()
- Withdraw()
- AddInterest()
- Transfer()
- SaveData()
- LoadData()

The AddCustomer(), UpdateCustomer(), and DeleteCustomer() methods are used by the main customer form. The AddAccountToCustomer() method creates a new account and links it to the selected customer. The Deposit(), Withdraw(), AddInterest(), and Transfer() methods are used by the managed accounts form.

The Transfer() method is one of the most important new additions. It checks the source account, target account, transfer amount, and same-account condition. Then it tries to withdraw from the source account first. If that works, it deposits into the target account. If it fails, the correct failed fee is applied based on account type and staff status.

The SaveData() and LoadData() methods were also added so the app can save and reopen customer and account data using XML serialization. This made the controller even more useful because it now handles both program actions and persistence.

One-to-Many Relationship

One of the main goals of this task was to support one-to-many relationship. This was done by storing a list of accounts inside the Customer class. So now one customer can own many accounts. This matches real banking more properly. It also makes the transfer feature possible, because the system can move money between the customer's own accounts. Without this change, the app would not be able to support the required account structure in a proper way.

Staff and General Customer Difference

Another main part of this task was to distinguish between general customers and bank staff. This was done using the isStaff attribute in the Customer class. The value is checked during failed transaction handling. When a failed transfer or failed withdrawal happens, a staff customer only pays half the normal fee.

Final Note:

Task 3 changed the old data model into a more complete banking model. The Customer class was extended to support multiple accounts. The Account class and its child classes were used to represent different banking rules. The BankController class was expanded so it can manage customers, accounts, transactions, transfers, and saved data.